HW02 Main Report - Domain Testing and Boundary Value Analysis

Student Information

- Student ID: 23127280
- Full name: Nguyen Hien Tuan Anh
- Class: 23KTPM2
- Repository: https://github.com/tnnhuaa/eshop-sut
- Report finalized: 2026-07-04
- SUT commit hash: `85af3ba875c88283615e22cb108f13e2fccaf0e9`

Scope

This report applies Domain Testing and Boundary Value Analysis to four selected EShop requirements.

Pool	Feature	Requirement ID	Platform	Current Status
A	Product Detail View	FR-06	Web User	Test design and manual execution completed
B	Order State Machine	FR-10	Web User + Web Admin	Frontend UI execution completed
C	Product Management CRUD	FR-15	Admin Web	Frontend UI execution completed
D	Product Listing and Search on Mobile	FR-05-M	React Native Mobile	Mobile UI execution completed

Method

The test design follows Domain Testing and Boundary Value Analysis principles. For each feature, I first identified the test basis, actors, preconditions, input variables, valid and invalid domains, requirement gaps, and assumptions. Boundary values were selected only when a requirement or reviewed assumption provided a boundary. Robustness values were separated from strict boundary values when the SRS did not define a limit.

AI was used to draft analysis artifacts, identify possible gaps, and structure test cases. All AI output is treated as draft material and must be reviewed against the SRS, ISTQB CTFL v4.0.1 concepts, and observed SUT behavior before being accepted.

FR-06 Product Detail View

Requirement Summary

FR-06 requires the Web User product detail page to display complete product information and allow the user to add a selected quantity to the cart. The main product information fields are image, name, price, description, and category. The quantity input must accept positive integers with minimum value `1`. A successful add-to-cart action should provide visible feedback, such as a cart count update or notification.

Supporting requirements affect this feature. FR-07 defines cart behavior when products are added. FR-21/FR-22 define GUI consistency, including price formatting and user-visible feedback.

Requirement Items

Requirement Item	Expected Behavior	Test Focus
Product image	Detail page displays a large product image	Existing image and broken image URL
Product name	Product name is visible	Normal and script-looking product names
Product price	Price is visible and formatted as money	Numeric/string price formatting with `₫` and thousands separators
Product description	Description is visible	Normal and HTML/script-looking descriptions
Product category	Category is visible to the user	Missing category display or category ID/name ambiguity
Quantity input	Accepts positive integers only	Empty, text, spaces, decimal, scientific notation, zero, negative, large values
Minimum quantity	Minimum valid value is `1`	Boundary values `0`, `1`, `2`
Add to cart	Adds selected product and quantity to cart	One-click behavior, repeated add behavior, cart row/quantity
Feedback	Shows visible success feedback	Toast, badge update, or cart state change

Requirement Gaps and Assumptions

Gap ID	Gap / Ambiguity	Testing Impact	Reviewed Assumption
FR06-GAP-01	Behavior for non-existing product ID is not defined	Invalid route/API cases need expected behavior	Page should show a clear product-not-found state and should not crash
FR06-GAP-02	Category display does not say category name or ID	Displaying only an ID may not be meaningful to users	Category should be human-readable
FR06-GAP-03	Decimal quantity handling is not specified	`1.5` may be accepted or parsed unexpectedly	Decimal quantity is invalid because requirement says positive integer
FR06-GAP-04	Empty, spaces, text, and scientific notation are not specified	Common invalid input classes could still reach cart logic	Non-integer and empty values should be rejected before adding to cart
FR06-GAP-05	No maximum quantity or stock rule is specified	Very large values may create unrealistic totals	Large quantity is a robustness/risk test, not a strict failure unless app breaks
FR06-GAP-06	Exact success feedback is flexible	Test must define acceptable evidence	Any visible confirmation or cart count update within a reasonable time is acceptable
FR06-GAP-07	Requirement does not explicitly say one click must add item	Ignored first click would affect user flow	One valid click should add the item and show feedback

Gap ID	Gap / Ambiguity	Testing Impact	Reviewed Assumption
FR06-GAP-08	Repeated add behavior from product detail is not explicit	Same product may duplicate rows instead of increasing quantity	Repeated add should increase existing cart quantity based on FR-07
FR06-GAP-09	Broken image behavior is not specified	Bad product data may break layout	Broken image should not crash or block page usage
FR06-GAP-10	Price formatting is defined globally, not inside FR-06	Product detail still needs consistent GUI formatting	Detail price should use `đ` and thousands separators
FR06-GAP-12	Safe display of HTML/script-looking product data is not explicit	Product data can come from admin-created content	React/UI should display content as text and not execute it

Domain Model

The main input domains are `product_id`, product data fields, `quantity`, and the add-to-cart action.

Variable	Valid Domain	Invalid / Risky Domain	Representative Values
`product_id`	Existing product ID	Non-existing, non-numeric, negative	`1`, `2`, `999999`, `abc`, `-1`
`product.imageUrl`	Reachable image URL	Broken URL	Valid seeded image, `https://invalid.invalid/image.png`
`product.name`	Non-empty product name	HTML/script-looking text	Normal name, `<script>alert(1)</script>`
`product.price`	Positive numeric value	Numeric string, missing/invalid formatting risk	Product 2 price display check
`product.description`	Text description	HTML/script-looking text	Normal text, `<b>bold</b><script>...`
`product.category`	Existing category shown clearly	Missing category, ID-only display	Expected human-readable category
`quantity`	Positive integer, minimum `1`	Empty, spaces, text, zero, negative, decimal, scientific notation, very large	`1`, `2`, `3`, empty, spaces, `abc`, `0`, `-1`, `1.5`, `1e2`, `999999999`

The selected domain values come from the user-visible contract of the product detail page. `product_id` is included because the detail page is route-driven: existing IDs cover the normal path, while non-existing, non-numeric, and negative IDs check whether the page fails safely. Image, name, price, description, and category are separated because FR-06 requires complete product information, and each display field can fail independently. Quantity receives the widest set of invalid classes because it is the only direct user input in this feature and has an explicit rule: positive integer with minimum `1`. Empty, whitespace, text, zero, negative, decimal, and scientific notation values were selected as separate equivalence classes because browser number inputs and JavaScript parsing can treat them differently.

Boundary Value Analysis

The explicit boundary in FR-06 is the minimum allowed quantity: `quantity >= 1`.

Boundary	Below Boundary	At Boundary	Above Boundary	Expected Result
Minimum positive integer quantity	`0`	`1`	`2`	`0` is rejected; `1` and `2` are accepted

Additional values such as `-1`, `1.5`, and `999999999` were included as negative-domain or robustness values. They are useful for finding defects but are not all official SRS boundaries.

The BVA values `0`, `1`, and `2` were chosen because they are immediately below, at, and immediately above the minimum valid quantity. This targets the most likely decision point in the input validation. `-1` extends the invalid below-boundary class to a clearly negative integer, while `1.5` and `1e2` check whether the implementation enforces the "integer" part of the rule instead of accepting numeric-looking values. `999999999` is kept as a robustness value because the SRS does not define stock or maximum quantity.

Test Design

FR-06 has 21 test cases:

Technique	Count	Coverage
Domain Testing	15	Product ID classes, product display fields, quantity invalid classes, add-to-cart feedback, duplicate add behavior, safe rendering, broken image URL
Boundary Value Analysis	6	Quantity values below, at, and above minimum; negative, decimal, and large robustness values

The detailed test cases are stored in `features/FR06_Product_Detail_Web/04-test-cases.csv`.

Manual Execution Summary

Result	Count
Executed	21
Passed	11
Failed	10
Blocked / Not Run	0

Main Defects Observed

Bug ID	Related Tests	Summary
BUG-FR06-001	FR06-DT-006, FR06-DT-015	Add to cart requires two clicks; the first click does not add the product or show successful feedback
BUG-FR06-002	FR06-DT-001	Product detail page does not display visible category information
BUG-FR06-003	FR06-DT-008	Adding the same product again creates duplicate cart rows instead of increasing the existing quantity

Bug ID	Related Tests	Summary
BUG-FR06-004	FR06-BVA-001	Quantity `0` is accepted and added to cart
BUG-FR06-005	FR06-BVA-004	Negative quantity is accepted and can create negative cart quantity/total
BUG-FR06-006	FR06-DT-010	Empty quantity is accepted and can create invalid cart data
BUG-FR06-007	FR06-DT-011	Whitespace/empty-after-space quantity can lead to invalid or `NaN` cart value
BUG-FR06-008	FR06-BVA-005	Decimal quantity is accepted and parsed to integer quantity `1` instead of rejected
BUG-FR06-009	FR06-DT-012	Scientific notation quantity such as `1e2` is accepted and parsed incorrectly instead of rejected

Detailed execution results are stored in `features/FR06_Product_Detail_Web/05-test-execution.md`.

FR-10 Order State Machine

Requirement Summary

FR-10 defines order states and allowed transitions. The selected feature is mainly a state-machine testing target, but it also contains domain testing aspects such as actor role, order ownership, target status, order ID validity, and invalid API input.

States and Expected Transitions

State	Meaning	Final?	Expected Outgoing Transitions
`pending`	Order placed, waiting for confirmation	No	`confirmed`, `canceled`
`confirmed`	Admin confirmed order	No	`shipping`, `canceled`
`shipping`	Order is being delivered	No	`delivered`
`delivered`	Order completed	Yes	None
`canceled`	Order canceled	Yes	None

Actors and Permissions

Actor	Expected Permission
User	Can cancel own order only in allowed states; cannot cancel in `shipping`; cannot modify another user's order
Admin	Can move orders through valid admin transitions and cancel where allowed
Unauthenticated user	Cannot change order state

Actor	Expected Permission
Non-admin authenticated user	Cannot use admin state-change APIs

Requirement Gaps and Assumptions

Gap ID	Gap / Ambiguity	Testing Impact	Reviewed Assumption
FR10-GAP-01	Admin cancellation from `shipping` is ambiguous	Changes whether `shipping -> canceled` by admin is valid	Treat as ambiguous and document execution result separately
FR10-GAP-02	Exact endpoints, request bodies, and response codes are not in SRS	Test cases need expected API results	Use API documentation/observed implementation; expect 2xx for valid transition and 4xx for invalid transition
FR10-GAP-03	Error message wording is not specified	Exact message assertions may be brittle	Require clear error, not exact wording
FR10-GAP-04	SRS does not say whether API can set arbitrary target state	Direct API may bypass UI restrictions	API must reject invalid target state
FR10-GAP-05	Non-existing order ID behavior is not specified	Needed for negative domain class	Return clear not-found style error and no state change
FR10-GAP-06	User modifying another user's order is not specified in FR-10	Security/ownership risk	User can only cancel own orders
FR10-GAP-07	Same-state transition behavior is not defined	Same-state updates can hide defects	Reject unless explicitly documented as idempotent
FR10-GAP-12	Final-state forbidden transitions are not listed exhaustively	AI may miss outgoing pairs from final states	Test representative transitions from both final states
FR10-GAP-13	Case sensitivity of status values is not defined	API may accept invalid variants	Status values should be lowercase enumerations only

Domain and Boundary Focus

FR-10 does not have a numeric boundary as its primary risk. The main test design should focus on transition classes and invalid domains.

Variable	Valid Domain	Invalid / Risky Domain
Current order state	`pending`, `confirmed`, `shipping`, `delivered`, `canceled`	Unknown/missing state
Target state	Valid next state for current state	Skipped state, backward state, final-state outgoing transition, same-state transition
Actor role	Owner user, admin	Unauthenticated user, non-owner user, non-admin user

Variable	Valid Domain	Invalid / Risky Domain
Order ID	Existing order owned/managed by actor	Non-existing, other user's order, malformed ID
API payload	Supported lowercase status	Unknown status, uppercase/mixed-case status, missing status

The FR-10 domains were selected around lifecycle behavior rather than form-field validation. `Current order state` and `target state` are the main variables because the same action can be valid or invalid depending on where the order currently is. `Actor role` and ownership are included because User and Admin have different permissions, and a transition that is valid for Admin may be invalid for a normal user. Order ID classes check target correctness and ownership protection. Payload/status variants are kept as design inputs because order states behave like enum values; accepting unknown, uppercase, mixed-case, or missing status values would allow behavior outside the intended state machine.

Boundary-style checks should be applied to order count or identifier classes only if supported by the implementation. For state behavior, the equivalent of a boundary is the edge between allowed and forbidden transitions, especially final states with no outgoing transition.

For FR-10, BVA is interpreted as transition-boundary analysis. The important edge is not a numeric value but the point where a transition changes from valid to invalid. For example, `confirmed -> canceled` is allowed for a user, while `shipping -> canceled` should be rejected. Final states such as `delivered` and `canceled` were selected because they are boundary states with no valid outgoing transitions. Repeated, skipped, backward, and same-state transitions were included to test the edges of the lifecycle model.

Recommended Test Coverage

FR-10 test design contains 32 test cases: 25 Domain Testing cases and 7 Boundary Value Analysis cases. Execution was performed from the Web User and Web Admin UI based on teacher clarification that functional testing should use the frontend. Result: 20 passed, 7 failed, and 5 blocked because those cases require API-only inputs that are not exposed by the UI.

FR-10 test design covers:

- Valid transition path: `pending -> confirmed -> shipping -> delivered`.
- Valid cancellation from `pending` and `confirmed`.
- Invalid skipped transitions such as `pending -> shipping` and `confirmed -> delivered`.
- Invalid backward transitions such as `confirmed -> pending`.
- Invalid outgoing transitions from `delivered` and `canceled`.
- Role/ownership checks for user cancellation and admin status updates.
- Invalid order ID and invalid status payload classes.
- Cross-role state consistency between User order history and Admin order management after cancellation or final-state actions.

Detailed artifacts are stored under `features/FR10_Order_State_Machine/`.

FR-10 Execution Findings

Bug ID	GitHub Issue	Related Tests	Summary
BUG-FR10-001	`#10`	FR10-DT-006, FR10-DT-021, FR10-BVA-002	Web User UI allows user to cancel an order while it is already `shipping`.
BUG-FR10-002	`#11`	FR10-DT-012, FR10-DT-021, FR10-DT-022, FR10-DT-023, FR10-BVA-004	Admin UI exposes `Đánh dấu Đã giao` for a canceled final-state order and can change it to delivered.

FR-15 Product Management CRUD

Requirement Summary

FR-15 requires Admin to create, view, update, and delete products. The key validation rules are product name, price, and category. It also requires that updates affect only the selected product. This feature combines CRUD workflow testing with domain testing for form/API inputs.

Requirement Items

Requirement Item	Expected Behavior	Test Focus
Create product	Admin can add a valid product	Valid name, price, category
View products	Admin can view product list	List refresh after operations
Update product	Admin can edit selected product	Only target product changes
Delete product	Admin can delete selected product	Deleted product no longer appears
Name required	Empty name rejected	Empty and spaces-only name
Name max length	Maximum 255 characters	254, 255, 256 characters
Price required	Missing price rejected	Empty/missing price
Price positive	Price must be greater than 0	`-1`, `0`, `0.01`, `1`
Category required	Existing category must be selected	Missing, invalid, deleted category ID

Requirement Gaps and Assumptions

Gap ID	Gap / Ambiguity	Testing Impact	Reviewed Assumption
FR15-GAP-01	FR-15 does not repeat admin authorization rules	Product write APIs may be exposed accidentally	Create/update/delete require admin authorization
FR15-GAP-02	Validation location is not specified	UI may block invalid input while API accepts it	Backend validation is required; frontend-only validation is insufficient
FR15-GAP-03	Whitespace-only name is not specified	`" "` may be accepted as non-empty	Trimmed empty name should be rejected
FR15-GAP-04	Length counting for Unicode/Vietnamese text is not defined	Boundary tests can be ambiguous	Use ASCII for length boundary tests
FR15-GAP-05	Price decimal handling is not specified	E-commerce price may be integer or decimal	Positive decimals are exploratory unless implementation rounds unexpectedly
FR15-GAP-06	No upper price bound is specified	Very large values can break display/calculation	Large price is robustness, not strict failure unless app breaks
FR15-GAP-07	`description` and `imageUrl` requiredness is not specified	UI fields exist but SRS validation focuses on name/price/category	Treat description and image URL as optional

Gap ID	Gap / Ambiguity	Testing Impact	Reviewed Assumption
FR15-GAP-08	Image URL validation is not specified	Broken image can affect listing/detail	Do not fail create solely for broken URL; verify display behavior separately
FR15-GAP-10	Non-existing product update/delete behavior is not specified	Negative API classes need expected results	Return clear error and no data change
FR15-GAP-13	Product list refresh behavior is not specified	UI may show stale data after CRUD action	List should reflect operation immediately or after clear reload
FR15-GAP-14	How to prove update isolation is not specified	Accidental update to other products is a high-risk regression	Compare before/after values for target and non-target products

Domain and BVA Focus

Variable	Valid Domain	Invalid / Risky Domain	Boundary / Representative Values
`name`	Non-empty string length 1..255	Empty, spaces-only, length 256+, script-looking text	1, 254, 255, 256 chars
`price`	Numeric value `> 0`	Missing, empty, zero, negative, non-numeric, very large	`-1`, `0`, `0.01`, `1`, `999999999999`
`category_id`	Existing category ID	Missing, non-existing, deleted, non-numeric	Existing ID, `999999`, empty
`description`	Optional text	Very long or script-looking text	Empty and long text
`imageUrl`	URL/path string	Empty, broken URL, non-image URL	Valid URL, broken URL
`product_id`	Existing product ID	Non-existing, deleted, malformed	Existing ID, `999999`
`actor_role`	Admin	User, unauthenticated	Admin token vs missing/user token

The FR-15 domains were selected from the CRUD workflow and the explicit input constraints. `name`, `price`, and `category_id` are primary variables because the SRS gives direct rules for them. `description` and `imageUrl` are still included because the Admin UI exposes them, but they are treated as optional/risky display data rather than strict validation fields. `product_id` is required for update/delete target correctness, and `actor_role` is included because product management belongs to the Admin subsystem. The representative values separate strict requirement failures from robustness risks: empty name, overlong name, zero price, and missing category are requirement-focused; broken image URL, very large price, and script-looking optional text are robustness or safe-display checks unless another requirement defines a hard rule.

The FR-15 BVA values follow the explicit boundaries in the requirement. Product name has a maximum length of 255, so `254`, `255`, and `256` check just below, at, and just above the boundary. Price must be greater than `0`, so `-1`, `0`, `0.01`, and `1` check invalid below/at the threshold and valid just above it. Category has no numeric range in the SRS, so its boundary is modeled as selection state: no category versus an existing category. Update isolation is treated as a decision boundary between the selected product and every non-target product, which is why the design compares target and non-target values before and after editing.

Test Coverage And Execution

FR-15 contains 48 test cases: 33 Domain Testing cases and 15 Boundary Value Analysis cases. Execution was performed from the Admin Web frontend. API-only invalid domains were marked as blocked when the Admin UI could not produce the required invalid category, missing `category_id`, non-existing product ID, or direct unauthenticated write condition.

FR-15 test coverage includes:

- Valid create, view, update, and delete flows.
- Required field validation for name, price, and category.
- Name length boundary values 254, 255, and 256.
- Price boundary values around `> 0`.
- Category missing and invalid category ID.
- Backend/API validation in addition to Admin UI validation.
- Authorization for unauthenticated and non-admin users.
- Update isolation by checking a non-target product remains unchanged.
- Deletion of non-existing product and stale-list behavior.

Manual Execution Summary

Result	Count
Executed	48
Passed	28
Failed	11
Blocked	9

Main Defects Observed

Bug ID	GitHub Issue	Related Tests	Summary
BUG-FR15-001	`#12`	FR15-DT-008	Whitespace-only product name is accepted.
BUG-FR15-002	`#13`	FR15-DT-025, FR15-BVA-007	Product can be created without price.
BUG-FR15-003	`#14`	FR15-BVA-008, FR15-BVA-009	Non-positive prices such as `-1` and `0` are accepted.
BUG-FR15-004	`#15`	FR15-BVA-006, FR15-BVA-015	Product name longer than 255 characters is accepted during create/update.
BUG-FR15-005	`#16`	FR15-DT-013, FR15-DT-019, FR15-DT-031, FR15-BVA-014	Updating one product temporarily changes all product names in Admin UI before reload.
REVIEW-FR15-006	`#17`	FR15-BVA-010	Decimal price `0.01` is accepted and displayed as `0,01 VND`; this passes current `> 0` wording but needs confirmation if VND price must be integer-only.

Execution Interpretation

The strongest FR-15 defects are missing validation for whitespace-only name, missing/invalid price values, overlong names, and immediate UI update isolation. The update-isolation finding is important because the requirement says only the edited product should change. In this implementation, persisted data appears correct after reload, but the immediate Admin UI state incorrectly shows many rows with the edited product name.

Blocked cases are not treated as missing work. They are valid domain cases that cannot be executed from the frontend UI under the teacher-confirmed functional testing scope.

Detailed artifacts are stored under `features/FR15_Product_CRUD_Admin/`.

FR-05-M Product Listing and Search on Mobile

Requirement Summary

FR-05-M adapts FR-05 Product Listing and Search to the React Native Mobile app. The mobile app should show product listing cards, product image/name/price, search by product name, loading state, empty state, and safe display of search keywords/product data.

Some FR-05 wording is web-specific, such as grid layout, alt text, and `<h1>`. For mobile testing, those are interpreted as mobile-equivalent UI and accessibility expectations rather than literal HTML requirements.

Requirement Items

Requirement Item	Expected Behavior	Mobile Interpretation
Product listing	Home page shows all products	Mobile shows product list/card layout
Product image	Each product shows image	Image is visible; accessibility label is optional unless specified
Product name	Product name is visible	Same as web
Product price	Price uses `đ` and thousands separators	Same as web
Search by name	Search filters by product name	Search input/button or equivalent mobile control
Safe keyword display	Keyword displayed safely, not as HTML	React Native text should display literal text
Loading state	Shows loading while fetching	Loading text or indicator
Empty state	Shows suitable message when no result	Clear no-result message
One `<h1>`	Web page heading rule	Not directly applicable to React Native

Requirement Gaps and Assumptions

Gap ID	Gap / Ambiguity	Testing Impact	Reviewed Assumption
FR05M-GAP-01	Grid and `<h1>` are web-specific	Literal web assertions would be invalid on mobile	Use mobile-equivalent layout/header checks
FR05M-GAP-02	Alt text is web-specific	Image accessibility may be missed	Check visible image; accessibility label is improvement unless required

Gap ID	Gap / Ambiguity	Testing Impact	Reviewed Assumption
FR05M-GAP-03	Search matching rules are not defined	Case, spaces, accents, and partial match may behave differently	Search should be case-insensitive substring search and trim spaces
FR05M-GAP-04	Empty search behavior is not defined	Empty input could mean all/no products or validation	Empty search should show default/all products
FR05M-GAP-05	Empty-state message content is not specified	Tests need expected no-result behavior	Any clear no-result message is acceptable
FR05M-GAP-06	Loading state timing is not specified	Loading may be too fast to observe	Loading indicator is expected during slow fetch; absence in fast fetch is hard to judge
FR05M-GAP-07	Network/API error behavior is not defined	Mobile backend/IP failures are common	App should show safe error and not crash
FR05M-GAP-10	Maximum search keyword length is not specified	Long input may break layout/query	Treat long keyword as robustness test
FR05M-GAP-11	HTML/script keyword behavior is not defined	Security-oriented domain class	Payload should be shown as literal text or safely handled
FR05M-GAP-12	HTML error response instead of JSON is not specified	Mobile app may crash or display raw HTML	App should show safe error and not crash
FR05M-GAP-15	Mobile backend configuration is not defined	LAN/emulator IP can fail across environments	Record actual environment in setup logs
FR05M-GAP-18	Returning to Home after search is not defined	Header/logo navigation may keep stale filtered results	Returning Home should show the default/all product listing

Domain and BVA Focus

Variable	Valid Domain	Invalid / Risky Domain	Representative Values
`search_keyword`	Empty, normal substring	Spaces-only, long text, special characters, HTML/script-looking text, Vietnamese accents	empty, `iphone`, `IPHONE`, `phone`, no-match keyword, `<script>alert(1)</script>`
`product_list`	Array of products	Empty array, malformed response, API error	Seeded list, no-result response
`product.name`	Displayable string	Empty, long, script-looking text	Normal seeded name, admin-created risky name
`product.price`	Positive numeric value	Missing, non-numeric, negative, numeric string	Seeded price and malformed test data if available
`product.imageUrl`	Reachable image URL	Empty, broken, slow image	Valid image, broken URL
`search_state`	Default/all products after Home navigation	Stale filtered result after header/logo navigation	Search first, then tap header/logo

Variable	Valid Domain	Invalid / Risky Domain	Representative Values
`network_state`	Backend reachable	Backend unreachable, wrong IP, timeout	Current LAN/emulator connection, backend stopped

The FR-05-M domains focus on observable mobile behavior. `search_keyword` is the main controllable input, so it includes normal text, case variations, leading/trailing spaces, Vietnamese accents, no-result text, long text, and HTML/script-looking strings. These values were selected because search defects often come from normalization, trimming, case sensitivity, encoding, and unsafe rendering. `product_list`, `product.name`, `product.price`, and `product.imageUrl` are included because every product card must display image, name, and formatted price. `search_state` was added after review because returning Home should restore the default listing, not preserve stale filtered results. `network_state` is included because mobile execution depends on backend reachability, emulator/LAN IP, and timeout behavior.

FR-05-M does not contain a clear numeric boundary in the SRS. Boundary-style testing should focus on keyword length only as robustness unless a maximum length is later specified.

The FR-05-M BVA set uses practical UI boundaries rather than formal numeric limits. An empty keyword is the lower boundary of search input and should return the default/all-product listing. A one-character keyword checks the smallest meaningful search text. A no-match keyword checks the boundary between result count `1` and result count `0`, where the empty-state requirement becomes visible. A many-match keyword checks the opposite display boundary where scrolling and card layout matter. The 256-character keyword is documented as robustness, not an official maximum, because the SRS does not define a keyword length limit.

Test Coverage And Execution

FR-05-M contains 25 test cases: 18 Domain Testing cases and 7 Boundary Value Analysis cases. Execution was performed from the React Native Mobile UI using the student's mobile environment. The execution result was recorded in `features/FR05_Product_Search_Mobile/Execution.xlsx`; remaining `TODO` actual results were interpreted as no issue observed based on the student's note.

FR-05-M test coverage includes:

- Default product listing on mobile.
- Product card image/name/price display.
- Search by exact, partial, lowercase/uppercase, and trimmed keyword.
- No-result keyword and empty-state message.
- Empty search returning default/all products.
- Spaces-only and very long keyword behavior.
- HTML/script-looking keyword displayed safely.
- Returning to Home from header/logo after a filtered search.
- Backend unreachable/API error state if feasible.
- Broken image or risky product data created through Admin/API if feasible.

Seed data for manual execution is prepared in `test-data/seed-fr05-mobile-products.js`. It creates stable `FR05M-` products for one-result, many-result, Vietnamese keyword, unsafe-looking name, and broken-image checks.

Manual Execution Summary

Result	Count
Executed	25
Passed	20

Result	Count
Failed	5
Blocked	0

Main Defects Observed

Bug ID	GitHub Issue	Related Tests	Summary
BUG-FR05M-001	`#18`	FR05M-DT-005	Search keyword with leading/trailing spaces is not trimmed, so it does not behave like the same valid keyword without spaces.
BUG-FR05M-002	`#19`	FR05M-DT-007, FR05M-BVA-003	No matching search result shows a blank result area instead of a suitable empty-state message.
BUG-FR05M-003	`#20`	FR05M-DT-018	Returning to Home from the header/logo keeps the previous search results instead of resetting to all products.
BUG-FR05M-004	`#21`	FR05M-BVA-006	A 256-character search keyword breaks the mobile result-label layout.

Detailed artifacts are stored under `features/FR05_Product_Search_Mobile/`.

Overall Summary

Metric	Count
Features selected	4
Features with requirement analysis completed	4
Features with test cases designed	4
Test cases designed	126
Test cases executed	126
Passed	79
Failed	33
Blocked / Not run	14
Defect IDs identified from executed features	20 defects and 1 price clarification

Artifact Index

Feature	Analysis Artifacts
FR-06	`features/FR06_Product_Detail_Web/01-requirement-analysis.md`, `02-domain-model.md`, `03-boundary-value-analysis.md`, `04-test-cases.csv`, `05-test-execution.md`, `06-ai-gap-analysis.md`
FR-10	`features/FR10_Order_State_Machine/01-requirement-analysis.md`, `02-state-transition-model.md`, `03-domain-model.md`, `04-boundary-value-analysis.md`, `05-test-cases.csv`, `06-test-execution.md`, `07-ai-gap-analysis.md`
FR-15	`features/FR15_Product_CRUD_Admin/01-requirement-analysis.md`, `02-domain-model.md`, `03-boundary-value-analysis.md`, `04-test-cases.csv`, `05-test-execution.md`, `06-ai-gap-analysis.md`
FR-05-M	`features/FR05_Product_Search_Mobile/01-requirement-analysis.md`, `02-domain-model.md`, `03-boundary-value-analysis.md`, `04-test-cases.csv`, `05-test-execution.md`, `06-ai-gap-analysis.md`

References

- EShop SRS in repository `README.md`
- ISTQB Foundation Level CTFL v4.0.1 study guide
- HW02 assignment brief